



Rapport de projet DevOps

Goat Calendar

Application de gestion de boards collaboratifs

Backend FastAPI, front web statique, MySQL et Docker Compose

Étudiants : Antonin Durand
Yann Renvoisé

Classe : Ingé2 LSI2

Année : 2025–2026

Date : Juin 2026

Lien du dépôt Git :

<https://github.com/GoatsSoftware/GoatCalendar>

Table des matières

1	Introduction	1
2	Synthèse des exigences	1
3	Architecture générale	1
3.1	Schéma d'architecture	2
4	Front web	2
4.1	Pages principales	2
4.2	Captures front de démonstration (3 pages)	3
5	Architecture en couches	5
6	Description des services backend	6
6.1	database_service	6
6.2	auth_service	6
6.3	board_service	6
6.4	shared_models	7
7	Modèle de données	7
8	Gestion de la concurrence	8
9	Docker et déploiement local	8
10	Pipeline DevOps et qualité	9
10.1	Gestion des dépendances	9
10.2	Qualité statique	9
10.3	Pipeline CI cible	10
10.4	Liens et résultats	10
11	Tests et couverture	12
12	Données seed	12
13	Sécurité et configuration	12
14	Limites actuelles	12
15	Conclusion	13
A	Annexe : Google Labs	13
B	Annexe : Sources Mermaid	14
B.1	Architecture	14
B.2	Couches	14
B.3	Modèle de données	15
B.4	Déploiement	16
B.5	Pipeline	16

1 Introduction

Goat Calendar est un projet DevOps réalisé en binôme. L'application permet de se connecter, consulter les boards disponibles, créer ou modifier des boards, afficher le détail d'un board, gérer ses rows, tasks, columns, comments, events et permissions.

Le sujet impose un dépôt Git, une pipeline CI, une architecture aback de type microservices en couche, au moins deux services back, des tests unitaires, d'intégration, une couverture de code d'au moins 80

2 Synthèse des exigences

Exigence	État dans Goat Calendar
Dépôt Git Pipeline CI	Le rapport contient le lien du dépôt GitHub du projet. Lance l'analyse SonarCloud, exécute les tests unitaires, d'intégration et de bout-en-bout, et publie dans PyPI les services Python et les images dans DockerHub.
Architecture en couches	Présente : routes FastAPI, services métier, repositories, schémas, models et DTO partagés.
Au moins deux services back	Présente : <code>auth_service</code> , <code>board_service</code> et <code>database_service</code> .
Intégration Docker	Présente : Docker Compose, trois images backend et un conteneur front Nginx.
Tests unitaires et mocks web	Présente : 93 fonctions de test Pytest réparties dans les packages backend.
Couverture de code	Sonar attend quatre rapports XML dans coverage-reports . Les fichiers de rapport ne sont pas versionnés dans la copie locale.
Qualité logicielle	Ruff configuré dans les services Python, et SonarCloud configuré à la racine.
Front web bonus	Présente : HTML, CSS et JavaScript natif servis par Nginx sur le port 8080.
Base de données bonus	Présente : MySQL 9.6.0, SQLAlchemy, aiomysql, seed automatique.
Google Labs	Emplacements réservés pour les captures d'écran démontrant la complétion des labs.

3 Architecture générale

Le projet est organisé autour de quatre blocs applicatifs :

- **front_service** : interface web statique servie par Nginx, pages login, dashboard et détail de board.
- **auth_service** : authentification OAuth2 password flow, émission et vérification de tokens JWT, gestion des utilisateurs.
- **board_service** : API principale des boards, colonnes, lignes, tâches, commentaires, événements et permissions.
- **database_service** : fournit le connecteur à la base de données, la création des tables à partir des schémas SQLAlchemy et le seeding d'un jeu de données de base.
- **shared_models** : schémas SQLAlchemy, DTO, enums, settings, exceptions et types communs aux services backend.

3.1 Schéma d'architecture

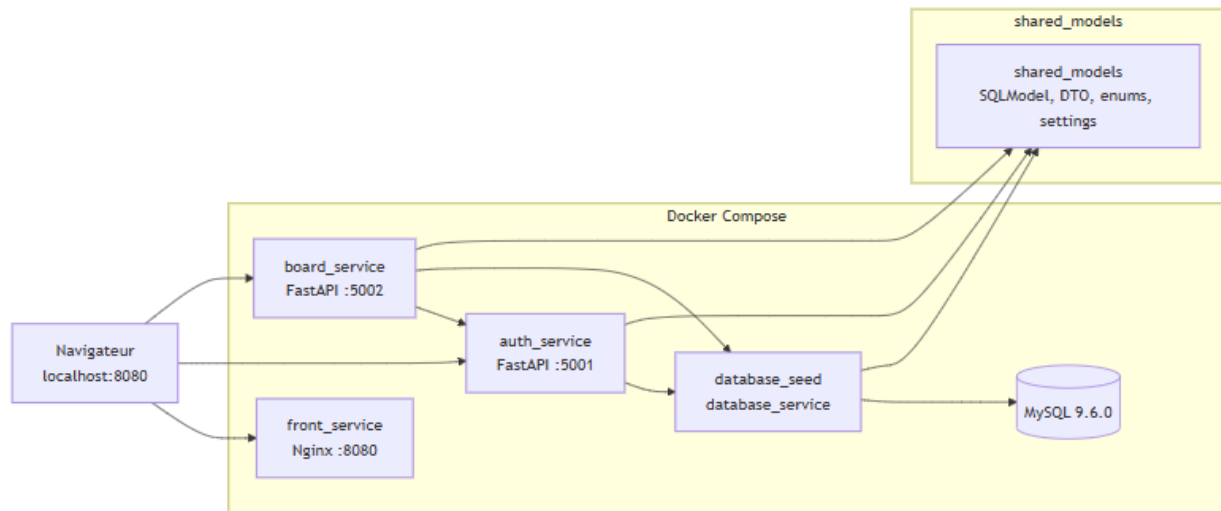


FIGURE 1 – Architecture applicative générée depuis Mermaid

4 Front web

Le front est volontairement simple et sans framework JavaScript. Il est composé de pages HTML, de CSS par responsabilité et de modules JavaScript natifs. En développement, il peut être servi avec `py -m http.server 8080`. En production Docker, il est copié dans `/usr/share/nginx/html` et servi par `nginx:1.27-alpine`.

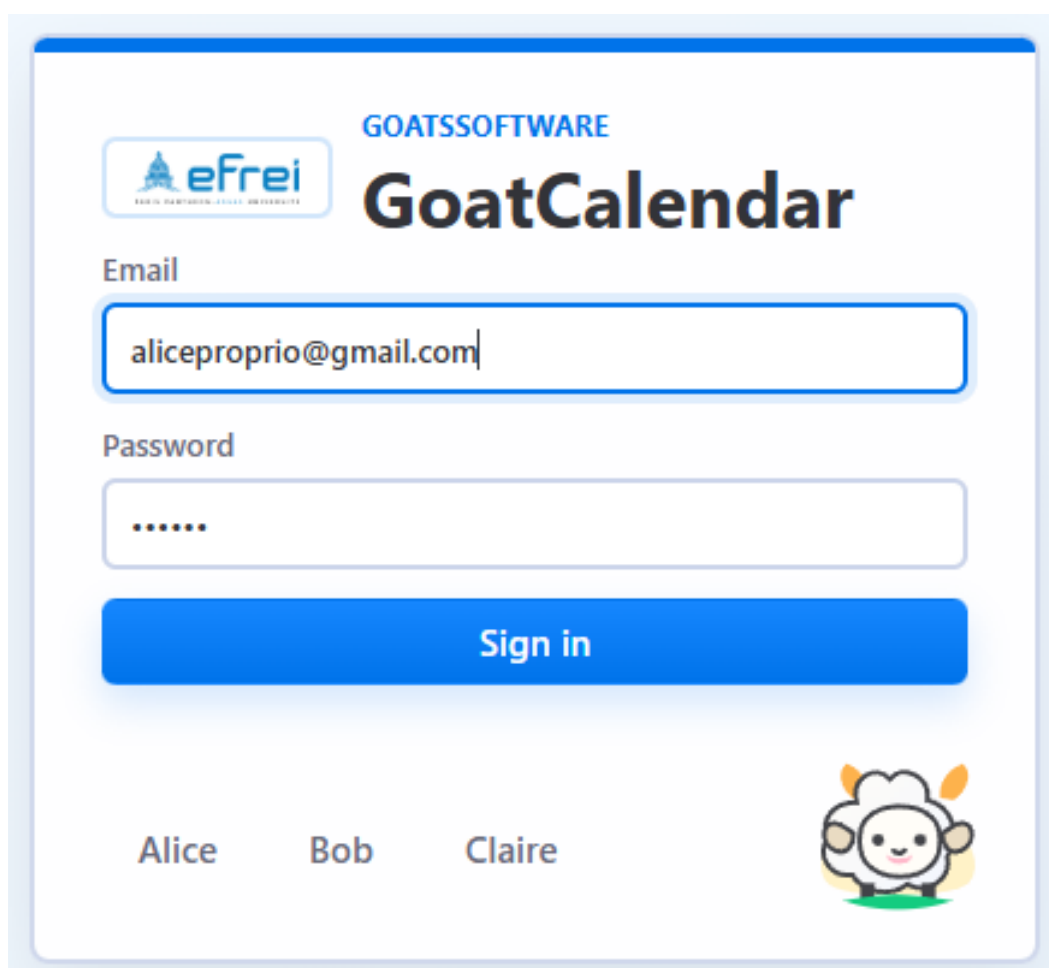
Les URLs des API sont centralisées dans `front/scripts/config.js` :

- Auth service : `http://localhost:5001`
- Board service : `http://localhost:5002`

4.1 Pages principales

- **Login page** : page `front/index.html`, formulaire email/password, boutons de comptes démo, récupération et stockage des tokens.
- **Dashboard page** : page `front/dashboard.html`, liste des boards, création de board, recherche utilisateurs, profil utilisateur.
- **Board page** : page `front/board.html`, chargement d'un board, affichage des rows et tasks, gestion columns, events et permissions.

4.2 Captures front de démonstration (3 pages)



The image shows a login page for 'GoatCalendar' by 'GOATSSOFTWARE'. At the top left is the 'efrei' logo (Ecole Française de Recherche en Informatique). The title 'GoatCalendar' is prominently displayed. Below the title, there are two input fields: 'Email' containing 'aliceproprio@gmail.com' and 'Password' with masked characters. A blue 'Sign in' button is positioned below the password field. At the bottom, there are three links: 'Alice', 'Bob', and 'Claire'. A cartoon goat icon is located in the bottom right corner.

GOATSSOFTWARE

GoatCalendar

Email

aliceproprio@gmail.com

Password

.....

Sign in

Alice Bob Claire



FIGURE 2 – Login page de Goat Calendar

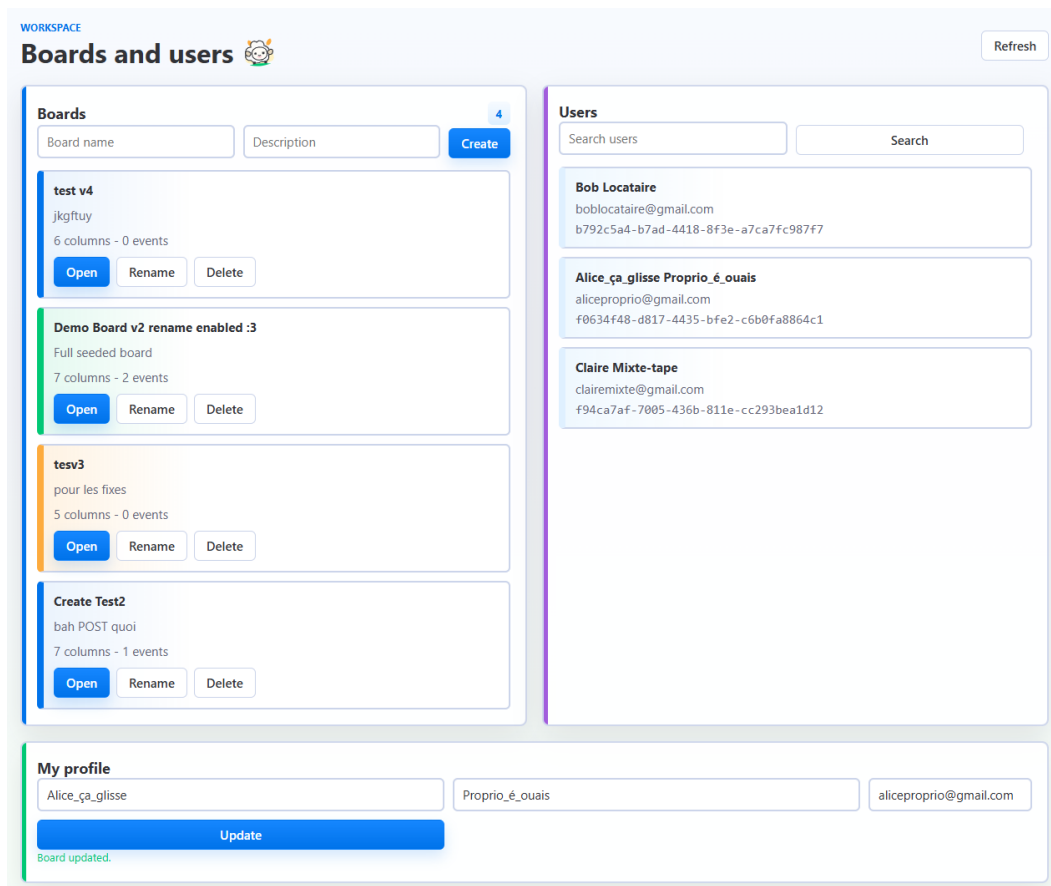


FIGURE 3 – Dashboard page avec boards, utilisateurs et profil

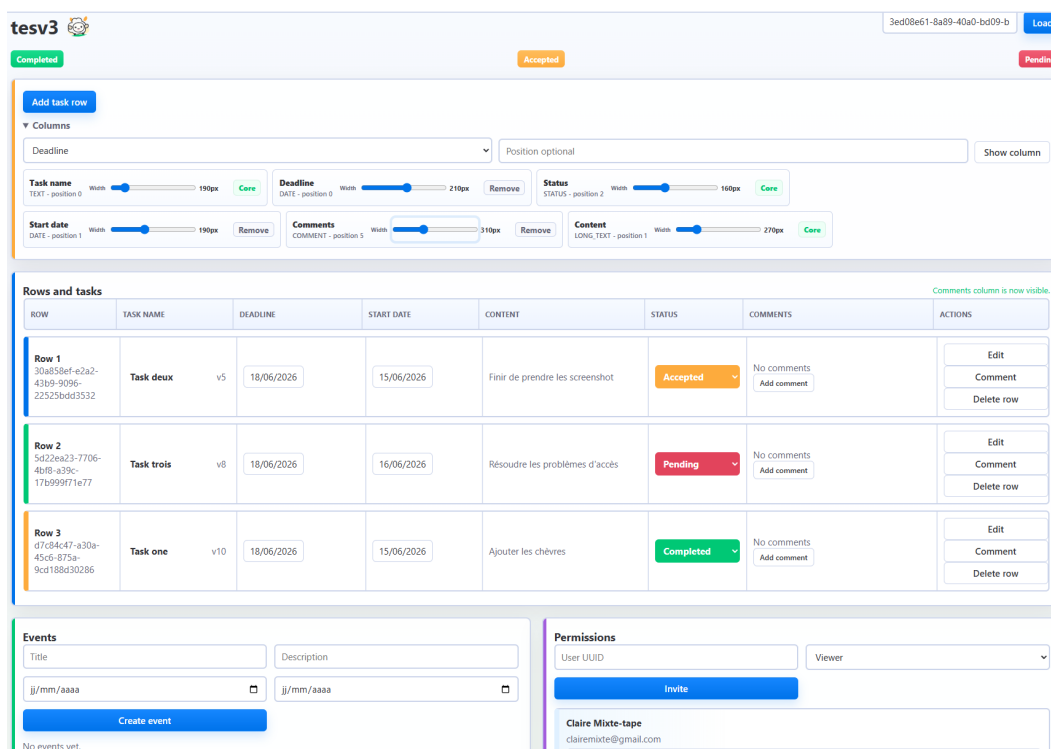


FIGURE 4 – Board page avec détail des rows d'un board

5 Architecture en couches

Les services backend suivent une architecture en couches conforme au sujet :

- **Controller/Web** : fichiers `routes/*.py`. Ils exposent les endpoints FastAPI, injectent les dépendances, appellent la couche logique pour traiter la demande, et convertissent les erreurs, ressources en réponses HTTP.
- **Services** : fichiers `services/*.py`. Ils portent la logique métier, la sérialisation DTO et les règles de création/mise à jour/récupération de ressource(s).
- **Data/Repositories** : fichiers `repositories/*.py` et `database.py`. Ils exécutent les requêtes SQLAlchemy.
- **Shared models** : fichiers `shared_models/*.py`. Il centralise la déclaration des schémas SQL-Model mappés aux tables SQL, modèles, énumérations, exceptions, DTO utilisés par les services back.

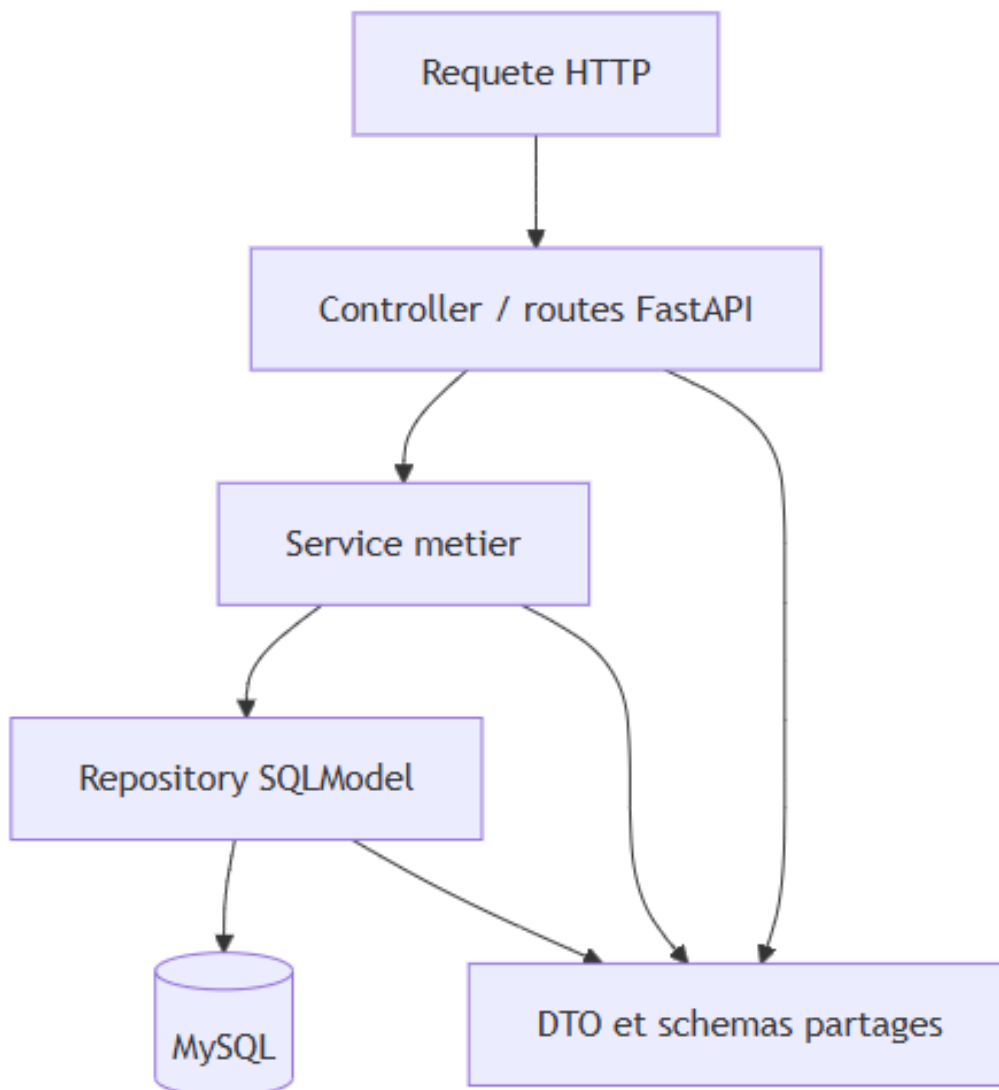


FIGURE 5 – Architecture en couches générée depuis Mermaid

6 Description des services backend

6.1 database_service

Le service `database_service` fournit le connecteur à la base de données, la création des tables à partir des schémas créés avec l'ORM `SQLModel` et le seeding d'un jeu de données de base. Dans Docker Compose, il est exécuté comme un conteneur court nommé `database_seed`. Son script principal lance `create_db_and_tables()`, puis `base_seeder()`.

La connexion utilise `SQLModel`, `SQLAlchemy async engine` et `aiomysql`. Une stratégie de retry attend la disponibilité de MySQL avec 10 tentatives espacées de 5 secondes.

6.2 auth_service

Le service d'authentification est une application FastAPI exposée sur le port 5001. Il dépend du package `database_service` pour effectuer les requêtes dans la base de données.

Il s'occupe aussi de l'insertion d'un jeu de données de base.

Routes principales :

- `POST /authentication/auth` : authentifie un utilisateur et retourne une paire de tokens JWT.
- `POST /authentication/refresh_access_token` : régénère un token d'accès à partir d'un refresh token.
- `GET /authentication/me` : retourne l'utilisateur connecté.
- `GET /users`, `GET /users/me`, `GET /users/search`, `PUT /users/me` : consultation et modification des utilisateurs.
- `GET /auth_monitoring/health` : healthcheck du service.

La sécurité repose sur `OAuth2PasswordBearer`, des mots de passe hashés avec `Argon2` via `Passlib`, et des tokens signés avec `python-jose`. Les durées de token sont configurées par variables d'environnement.

6.3 board_service

Le service Board est une application FastAPI exposée sur le port 5002. Il contient le coeur métier de l'application, à savoir la gestion des boards.

Il dépend du package `database_service` via la dépendance `get_db_session`, pour effectuer les requêtes dans la base de données. Il dépend du package `auth_service` via la dépendance `get_current_user`, pour sécuriser les endpoints et vérifier de l'authenticité de l'utilisateur d'après le token JWT transmis dans le header de la requête HTTP.

Routes principales :

- `/boards` : CRUD des boards et recherche des boards d'un utilisateur.
- `/board_columns` : colonnes fonctionnelles d'un board.
- `/board_rows` : lignes horizontales d'un board.
- `/board_row_tasks` : tâches rattachées à une ligne et à une colonne.
- `/board_row_comments` : commentaires rattachés à une ligne.
- `/board_events` : événements et jalons d'un board.
- `/boards/{board_id}/permissions` : rôles utilisateurs sur un board.
- `/board_monitoring/health` : healthcheck du service.

6.4 shared_models

Ce package contient les différentes classes utilisés par les différents services back :

- tables : `User`, `Board`, `BoardColumn`, `BoardRow`, `BoardRowTask`, `BoardRowComment`, `BoardEvent`, `UserBoardPermission` ;
- DTO d'entrée et de sortie pour les endpoints ;
- enums : rôles, statuts, types de colonnes, type de token, statut de santé ;
- settings : configuration serveur, base de données et authentification.

7 Modèle de données

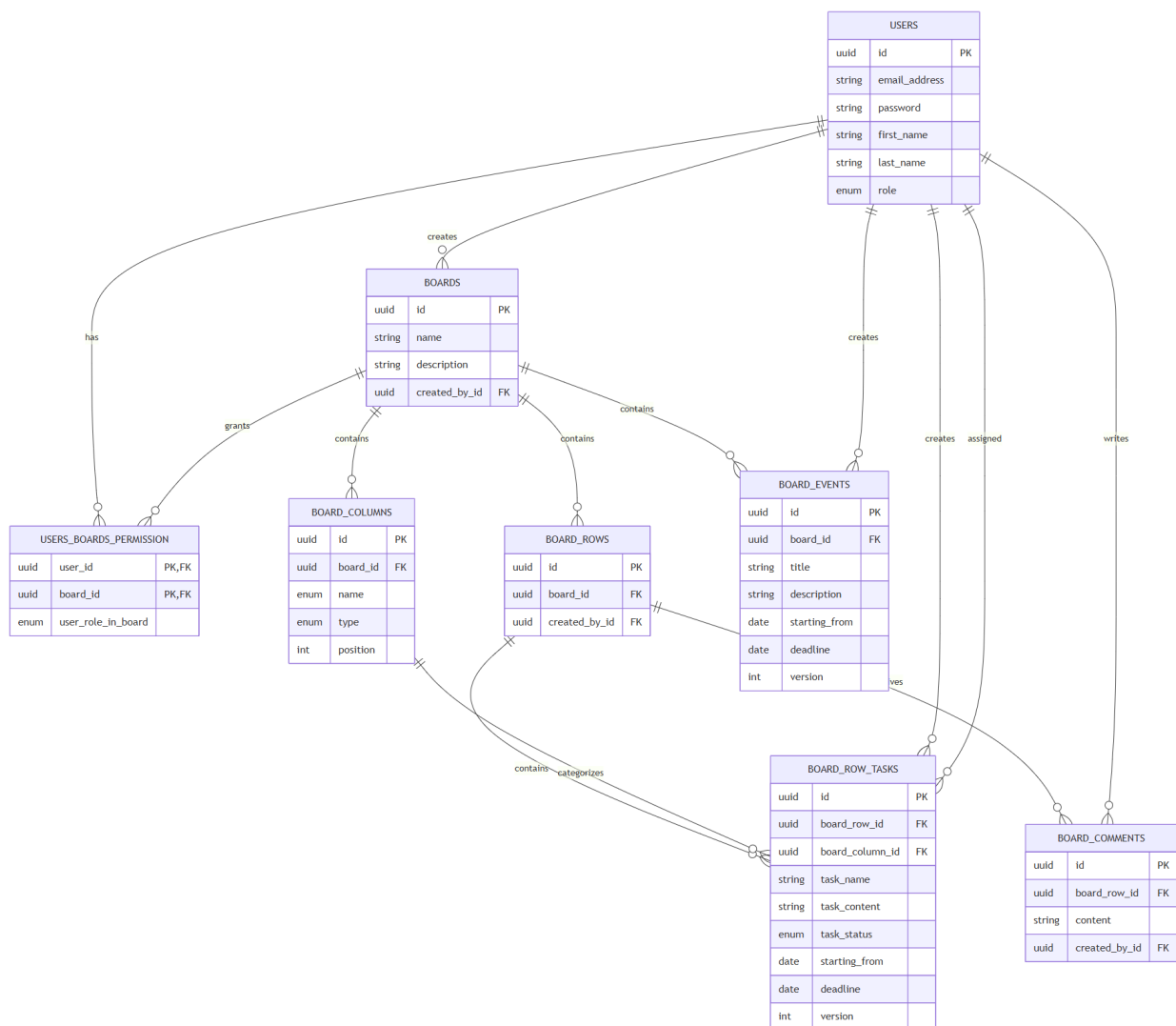


FIGURE 6 – Modèle relationnel principal généré depuis Mermaid

Table	Rôle
users	Identité applicative, email unique, mot de passe hash, prénom, nom, rôle global ADMIN ou USER.
boards	Tableau collaboratif avec nom, description, créateur et time-stamps.

<code>users_boards_permission</code>	Table pivot entre utilisateur et board, avec rôle OWNER, EDITOR ou VIEWER.
<code>board_columns</code>	Colonnes configurables du board : TaskID, TaskName, TaskContent, TaskStatus, Deadline, etc.
<code>board_rows</code>	Ligne horizontale d'un board, support des tâches et commentaires.
<code>board_row_tasks</code>	Tâche positionnée dans une ligne et une colonne, avec statut, dates, assignation et version.
<code>board_comments</code>	Commentaires sur une ligne de board.
<code>board_events</code>	Événements de calendrier liés à un board, avec dates et version.

8 Gestion de la concurrence

Le code implémente un verrouillage optimiste sur les tâches via le champ `version`. Lors d'une mise à jour de `BoardRowTask`, le repository exécute une mise à jour conditionnelle :

Listing 1 – Principe du verrouillage optimiste

```
UPDATE board_row_tasks
SET ..., version = version + 1
WHERE id = :task_id
AND version = :version_from_client;
```

Si aucune ligne n'est modifiée, le repository retourne `None`. La couche service transforme ce résultat en exception de concurrence, puis la route renvoie une erreur HTTP adaptée. Cela évite qu'un utilisateur écrase silencieusement la modification d'un autre utilisateur.

9 Docker et déploiement local

Le fichier `docker-compose.yml` orchestre cinq conteneurs :

- `mysql_db` : image `mysql:9.6.0`, port 3306, volume persistant `mysql_data`, healthcheck `mysqladmin ping`.
- `database_seed` : image `antonind5/goat-calendar-database_service`, attend que MySQL soit sain, puis se termine.
- `auth_service` : image `antonind5/goat-calendar-auth_service`, port 5001, `FRONT_URL=http://localhost:8080`.
- `board_service` : image `antonind5/goat-calendar-board_service`, port 5002, `FRONT_URL=http://localhost:8080`.
- `front_service` : build local depuis `production/Dockerfile.FrontService`, port 8080 vers Nginx port 80.

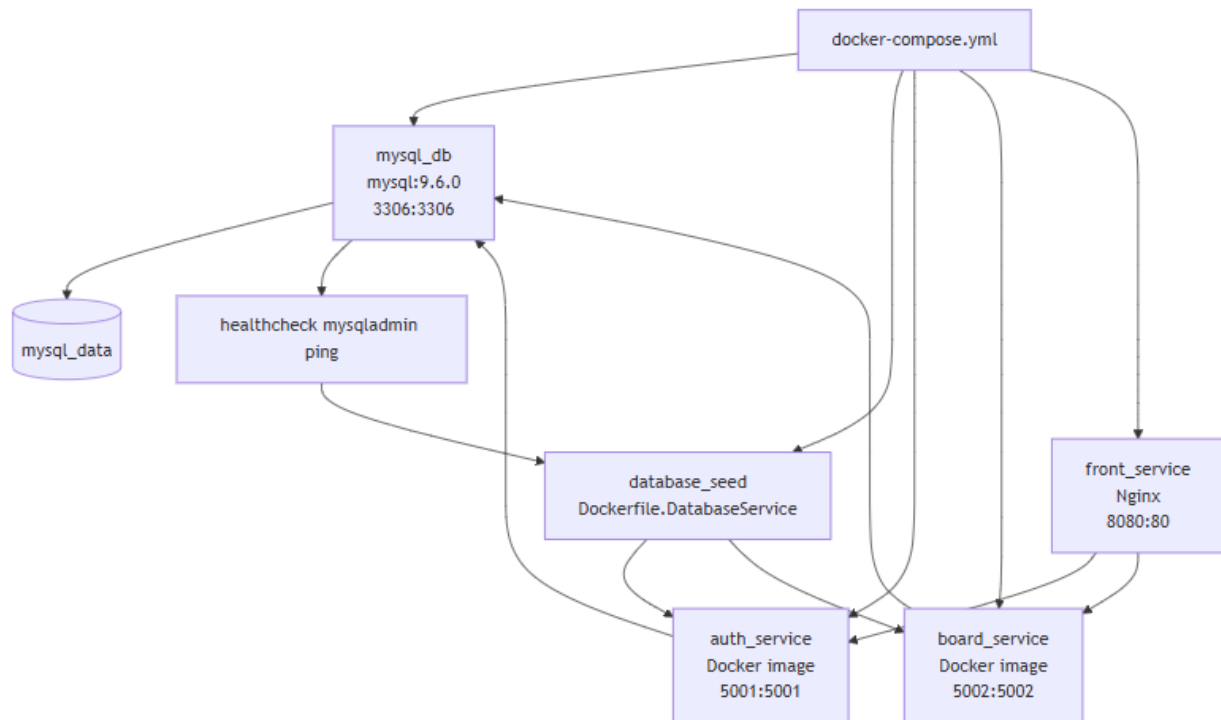


FIGURE 7 – Déploiement Docker Compose généré depuis Mermaid

10 Pipeline DevOps et qualité

10.1 Gestion des dépendances

Chaque package Python utilise PDM avec `pyproject.toml` et `pdm.lock`. Les dépendances locales entre services sont relatives en développement. Les dépendances de production sont exportées dans les Dockerfiles avec `pdm export`. Le front ne nécessite pas de build Node : il est copié tel quel dans l'image Nginx.

10.2 Qualité statique

Ruff est utilisé par tous les packages back Python pour le formatage et le contrôle de la qualité du code. Les scripts PDM pour lancer diverses commandes ruff sont les suivantes :

- `ruff_format` : formatage ;
- `ruff_lint` : vérification et correction automatique ;
- `ruff_all` : exécution combinée.

SonarCloud est configuré à la racine avec :

- organisation : `goatssoftware` ;
- project key : `GoatsSoftware_GoatCalendar` ;
- exclusion du front dans l'analyse Sonar Python ;
- chemins de couverture : `coverage-reports/coverage-database_service.xml`, `coverage-shared_models.xml`, `coverage-auth_service.xml`, `coverage-board_service.xml`.

10.3 Pipeline CI cible

Le diagramme ci-dessous décrit la pipeline cible pour automatiser les étapes déjà préparées : installation Python 3.13, PDM, lint Ruff, tests Pytest, couverture XML, build Docker et analyse SonarCloud.

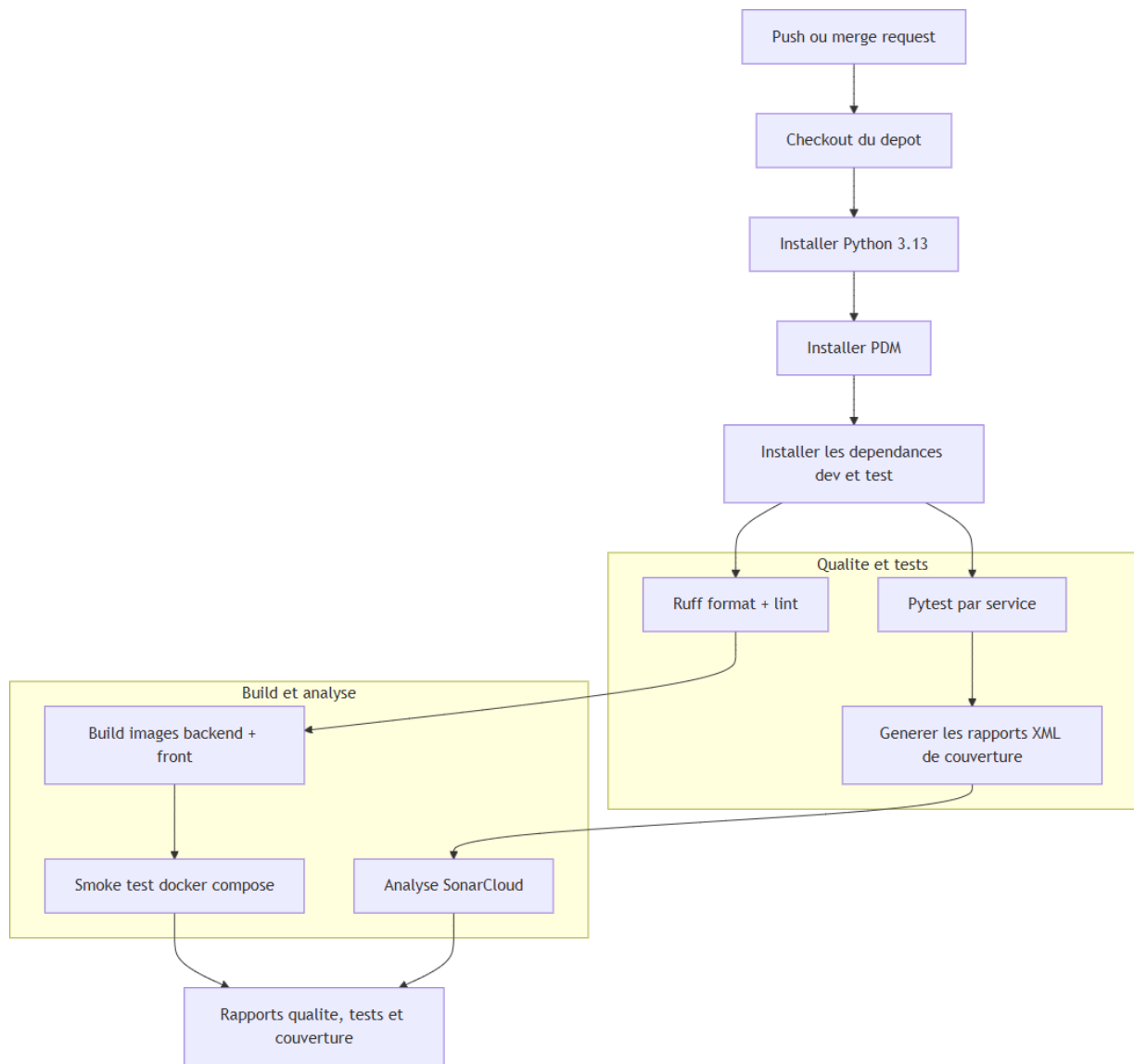


FIGURE 8 – Pipeline CI cible générée depuis Mermaid

10.4 Liens et résultats

Les résultats de la qualité et de l'intégration continue sont consultables sur les plateformes suivantes :

- SonarCloud : https://sonarcloud.io/project/overview?id=GoatsSoftware_GoatCalendar
- GitHub Actions : <https://github.com/GoatsSoftware/GoatCalendar/actions>

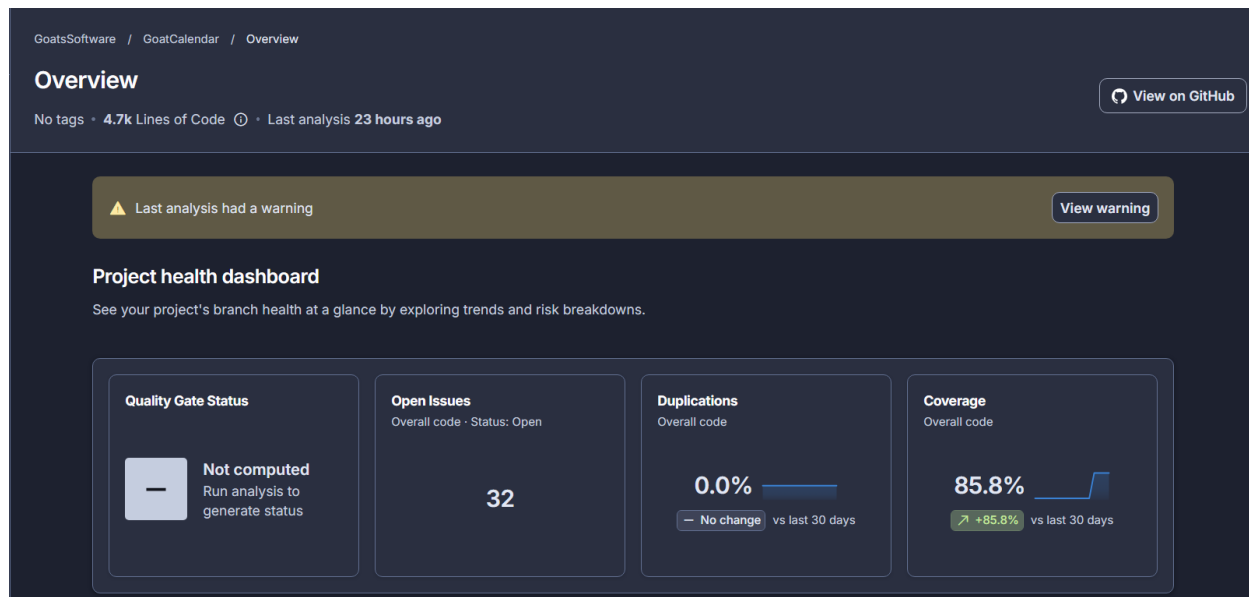


FIGURE 9 – Résultats SonarCloud du projet Goat Calendar

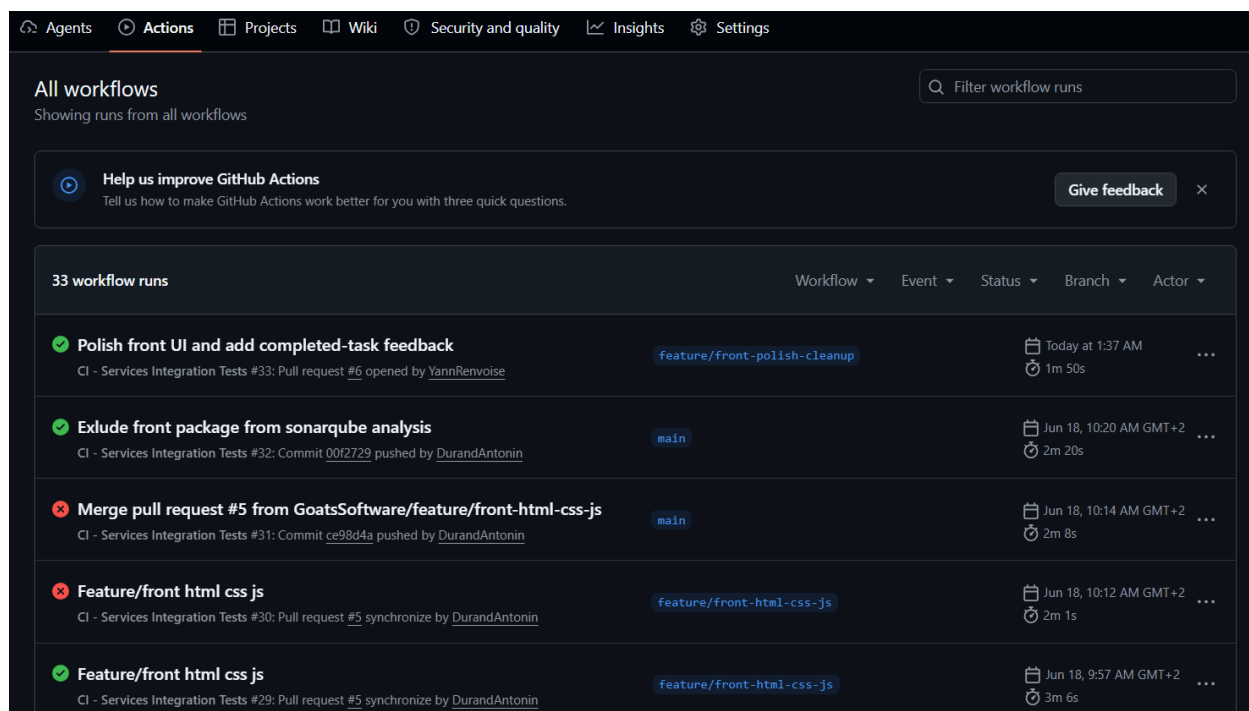


FIGURE 10 – Résultats GitHub Actions du projet Goat Calendar

11 Tests et couverture

La version finale contient plus de 100 tests Pytest, réparties dans les quatre packages backend :

- `auth_service/tests` : routes d'authentification, services, repository utilisateur, hash et JWT.
- `board_service/tests` : services, repositories, routes boards, rows et tasks.
- `database_service/tests` : database, seeder et main.
- `shared_models/tests` : settings, schémas, enums et DTO.

Les dépendances de test incluent `pytest` et `pytest-cov`. Les rapports de couverture attendus par SonarCloud sont séparés par package dans `coverage-reports`.

12 Données seed

Le seed crée un jeu de données de base pour permettre d'utiliser l'application directement :

- trois utilisateurs : Alice Proprio, Bob Locataire, Claire Mixte ;
- un board : `Demo Board` ;
- cinq colonnes : `TaskID`, `TaskName`, `TaskContent`, `TaskStatus`, `Deadline` ;
- cinq lignes ;
- vingt-cinq tâches, soit cinq tâches par ligne ;
- un commentaire par ligne ;
- deux événements : `Kickoff` et `Mid Review` ;
- deux permissions explicites : un `EDITOR` et un `VIEWER`.

Les comptes de démo utilisés par le front sont :

- `aliceproprio@gmail.com`
- `boblocataire@gmail.com`
- `clairemixte@gmail.com`

Le mot de passe de démonstration est `azerty`.

13 Sécurité et configuration

Les services utilisent CORS avec `FRONT_URL`, OAuth2 Bearer token, JWT et hash Argon2. Les variables d'environnement de production sont partiellement injectées dans `docker-compose.yml`. Les identifiants MySQL et certaines clés de démonstration restent visibles dans les fichiers de configuration et le README. Pour une production réelle, ces valeurs devraient être déplacées vers des secrets Docker, des variables CI/CD ou des fichiers `.env` non versionnés.

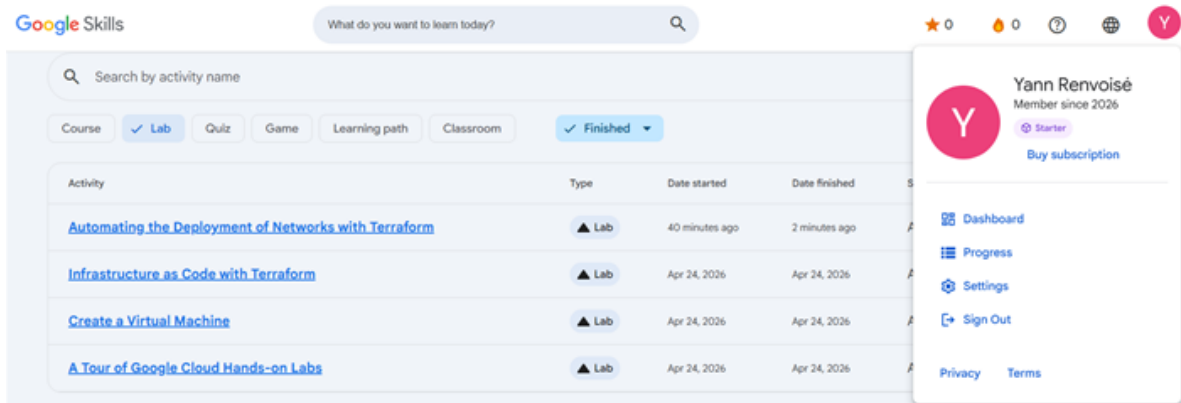
14 Limites actuelles

- Le front est statique et ne contient pas de tests automatisés JavaScript.
- Les secrets de démonstration sont visibles dans les fichiers de configuration.
- Certains docstrings backend semblent provenir d'un ancien domaine métier et devraient être harmonisés avec Goat Calendar.

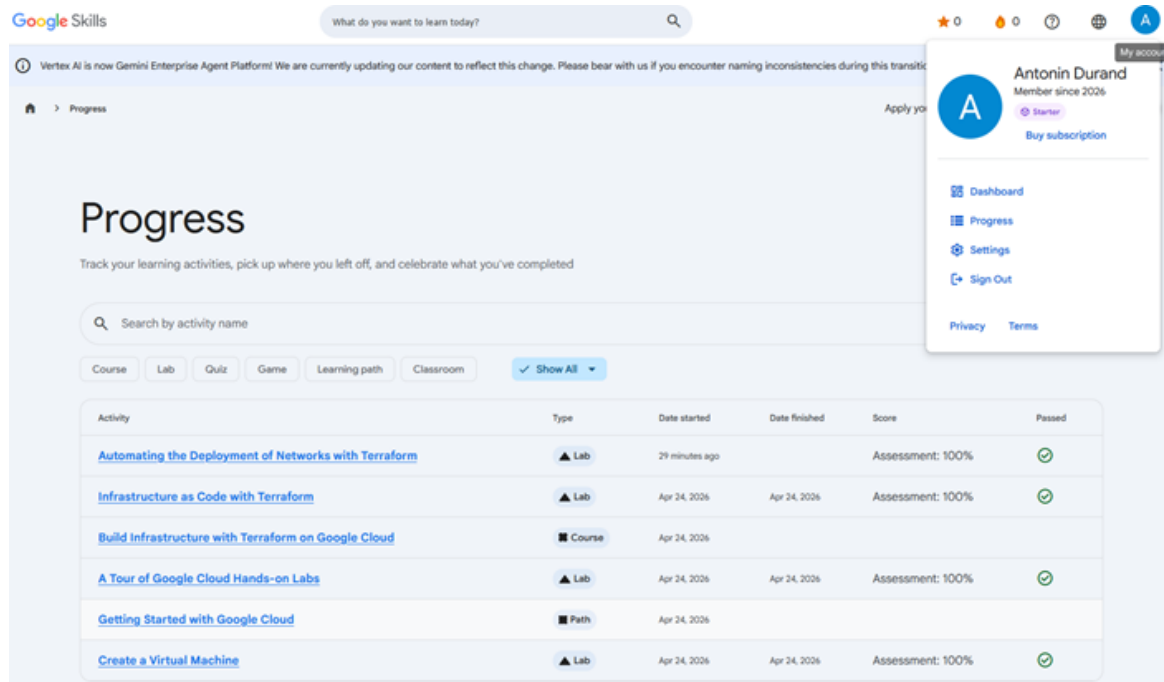
15 Conclusion

Goat Calendar présente une base DevOps complète pour le cadre du projet : architecture microservices en couches pour la partie back, un front bonus, une base de données MySQL, PyPi, DockerHub et Docker Compose pour le déploiement, des tests Pytest, Ruff et configuration Sonar-Cloud. Le projet respecte les attentes principales du sujet et ajoute les deux bonus importants : interface web et base de données.

A Annexe : Google Labs



Yann Renvoisé



Antonin Durand

FIGURE 11 – Captures Google Labs

B Annexe : Sources Mermaid

B.1 Architecture

```
flowchart LR
    browser[Navigateur<br/>localhost:8080]

    subgraph docker[Docker Compose]
        mysql[(MySQL 9.6.0)]
        seed[database_seed<br/>database_service]
        auth[auth_service<br/>FastAPI :5001]
        board[board_service<br/>FastAPI :5002]
        front[front_service<br/>Nginx :8080]
    end

    subgraph shared[shared_models]
        models[shared_models<br/>SQLModel, DTO, enums, settings]
    end

    browser --> front
    browser --> auth
    browser --> board
    board --> auth
    auth --> seed
    board --> seed
    seed --> mysql

    auth --> models
    board --> models
    seed --> models
```

B.2 Couches

```
flowchart TD
    http[Requete HTTP]
    route[Controller / routes FastAPI]
    service[Service metier]
    repo[Repository SQLModel]
    db[(MySQL)]
    dto[DTO et schemas partages]

    http --> route
    route --> service
    service --> repo
    repo --> db

    route --> dto
    service --> dto
    repo --> dto
```


B.3 Modèle de données

```
erDiagram
    USERS ||--o{ BOARDS : creates
    USERS ||--o{ USERS_BOARDS_PERMISSION : has
    BOARDS ||--o{ USERS_BOARDS_PERMISSION : grants
    BOARDS ||--o{ BOARD_COLUMNS : contains
    BOARDS ||--o{ BOARD_ROWS : contains
    BOARDS ||--o{ BOARD_EVENTS : contains
    BOARD_ROWS ||--o{ BOARD_ROW_TASKS : contains
    BOARD_ROWS ||--o{ BOARD_COMMENTS : receives
    BOARD_COLUMNS ||--o{ BOARD_ROW_TASKS : categorizes
    USERS ||--o{ BOARD_ROW_TASKS : creates
    USERS ||--o{ BOARD_ROW_TASKS : assigned
    USERS ||--o{ BOARD_COMMENTS : writes
    USERS ||--o{ BOARD_EVENTS : creates

    USERS {
        uuid id PK
        string email_address
        string password
        string first_name
        string last_name
        enum role
    }

    BOARDS {
        uuid id PK
        string name
        string description
        uuid created_by_id FK
    }

    USERS_BOARDS_PERMISSION {
        uuid user_id PK,FK
        uuid board_id PK,FK
        enum user_role_in_board
    }

    BOARD_COLUMNS {
        uuid id PK
        uuid board_id FK
        enum name
        enum type
        int position
    }

    BOARD_ROWS {
        uuid id PK
        uuid board_id FK
        uuid created_by_id FK
    }

    BOARD_ROW_TASKS {
        uuid id PK
        uuid board_row_id FK
        uuid board_column_id FK
        string task_name
        string task_content
    }
```

```

    enum task_status
    date starting_from
    date deadline
    int version
}

BOARD_COMMENTS {
    uuid id PK
    uuid board_row_id FK
    string content
    uuid created_by_id FK
}

BOARD_EVENTS {
    uuid id PK
    uuid board_id FK
    string title
    string description
    date starting_from
    date deadline
    int version
}

```

B.4 Déploiement

```

flowchart TD
    compose[docker-compose.yml]

    compose --> mysql[mysql_db<br/>mysql:9.6.0<br/>3306:3306]
    compose --> seed[database_seed<br/>Dockerfile.DatabaseService]
    compose --> auth[auth_service<br/>Docker image<br/>5001:5001]
    compose --> board[board_service<br/>Docker image<br/>5002:5002]
    compose --> front[front_service<br/>Nginx<br/>8080:80]

    mysql --> volume[(mysql_data)]
    mysql --> health[healthcheck mysqladmin ping]

    health --> seed
    seed --> auth
    seed --> board

    auth --> mysql
    board --> mysql
    front --> auth
    front --> board

```

B.5 Pipeline

```

flowchart TD
    push[Push ou merge request]
    checkout[Checkout du depot]
    python[Installer Python 3.13]
    pdm[Installer PDM]
    deps[Installer les dependances dev et test]

    subgraph qualite[Qualite et tests]
        ruff[Ruff format + lint]
    end

```

```
    tests[Pytest par service]
    coverage[Generer les rapports XML de couverture]
end

subgraph livraison[Build et analyse]
    docker[Build images backend + front]
    compose[Smoke test docker compose]
    sonar[Analyse SonarCloud]
end

result[Rapports qualite, tests et couverture]

push --> checkout --> python --> pdm --> deps
deps --> ruff
deps --> tests
tests --> coverage
ruff --> docker --> compose
coverage --> sonar
compose --> result
sonar --> result
```